

```

import React, {
  createContext,
  useContext,
  useState,
  useEffect,
  useRef,
  useCallback,
  useMemo,
} from "react";
import { X, Plus } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// 🏠 LOCAL TYPE ISOLATION GATEWAY
export interface TabsRootProps extends React.HTMLAttributes<HTMLDivElement> {
  defaultValue: string;
  value?: string;
  onValueChange?: (value: string) => void;
  children: React.ReactNode;
  orientation?: "horizontal" | "vertical";
  activationMode?: "automatic" | "manual";
}

export interface TabsListProps extends React.HTMLAttributes<HTMLDivElement> {
  children: React.ReactNode;
  showAddButton?: boolean;
  onAddTab?: () => void;
}

export interface TabsTriggerProps extends
  React.ButtonHTMLAttributes<HTMLButtonElement> {
  value: string;
  children: React.ReactNode;
  disabled?: boolean;
  isCloseable?: boolean;
  onClose?: (value: string) => void;
}

export interface TabsContentProps extends React.HTMLAttributes<HTMLDivElement> {
  value: string;
  children: React.ReactNode;
  forceMount?: boolean;
}

interface TabsContextValue {

```

```

value: string;
onValueChange: (value: string) => void;
orientation: "horizontal" | "vertical";
activationMode: "automatic" | "manual";
registerTrigger: (value: string, ref: HTMLButtonElement | null) => void;
unregisterTrigger: (value: string) => void;
triggerRefs: Map<string, HTMLButtonElement | null>;
activeTriggerRef: React.MutableRefObject<HTMLButtonElement | null>;
setActiveTriggerRef: (ref: HTMLButtonElement | null) => void;
}

const TabsContext = createContext<TabsContextValue | null>(null);

function useTabsContext() {
  const context = useContext(TabsContext);
  if (!context) {
    throw new Error("Tabs compound components must be used within TabsRoot");
  }
  return context;
}

// 🌀 TABS ROOT - State Management & Context Provider
export const TabsRoot = React.forwardRef<HTMLDivElement, TabsRootProps>(
  (
    {
      defaultValue,
      value: controlledValue,
      onValueChange,
      children,
      orientation = "horizontal",
      activationMode = "automatic",
      className,
      ...props
    },
    ref,
  ) => {
    const [uncontrolledValue, setUncontrolledValue] = useState(defaultValue);
    const isControlled = controlledValue !== undefined;
    const currentValue = isControlled ? controlledValue : uncontrolledValue;

    const triggerRefs = useRef<Map<string, HTMLButtonElement | null>>(
      new Map(),
    );
    const activeTriggerRef = useRef<HTMLButtonElement | null>(null);
    const [indicatorStyle, setIndicatorStyle] = useState<React.CSSProperties>(
      {},
    );

    const registerTrigger = useCallback(
      (value: string, ref: HTMLButtonElement | null) => {

```

```

        triggerRefs.current.set(value, ref);
    },
    [],
);

const unregisterTrigger = useCallback((value: string) => {
    triggerRefs.current.delete(value);
}, []);

const setActiveTriggerRef = useCallback((ref: HTMLButtonElement | null) => {
    activeTriggerRef.current = ref;
}, []);

const handleValueChange = useCallback(
    (value: string) => {
        if (!isControlled) {
            setUncontrolledValue(value);
        }
        onValueChange?.(value);
    },
    [isControlled, onValueChange],
);

const updateIndicatorPosition = useCallback(() => {
    const activeTrigger = activeTriggerRef.current;
    const tabsList = activeTrigger?.closest(
        "[data-tabs-list]",
    ) as HTMLElement | null;

    if (activeTrigger && tabsList) {
        const triggerRect = activeTrigger.getBoundingClientRect();
        const listRect = tabsList.getBoundingClientRect();

        if (orientation === "horizontal") {
            setIndicatorStyle({
                width: `${triggerRect.width}px`,
                transform: `translateX(${triggerRect.left - listRect.left}px)`,
                height: "2px",
            });
        } else {
            setIndicatorStyle({
                height: `${triggerRect.height}px`,
                transform: `translateY(${triggerRect.top - listRect.top}px)`,
                width: "2px",
            });
        }
    }
}, [orientation]);

useEffect(() => {

```

```

    updateIndicatorPosition();
    window.addEventListener("resize", updateIndicatorPosition);
    return () =>
      window.removeEventListener("resize", updateIndicatorPosition);
  }, [currentValue, updateIndicatorPosition]);

const contextValue = useMemo<TabsContextValue>(
  () => ({
    value: currentValue,
    onValueChange: handleValueChange,
    orientation,
    activationMode,
    registerTrigger,
    unregisterTrigger,
    triggerRefs: triggerRefs.current,
    activeTriggerRef,
    setActiveTriggerRef,
  })),
[
  currentValue,
  handleValueChange,
  orientation,
  activationMode,
  registerTrigger,
  unregisterTrigger,
  setActiveTriggerRef,
],
);

return (
  <TabsContext.Provider value={contextValue}>
    <div
      ref={ref}
      className={cn("relative", className)}
      data-tabs-root
      {...props}
    >
      {children}
      <div
        className={cn(
          "absolute bg-brand-primary transition-all duration-300 ease-out
pointer-events-none",
          orientation === "horizontal" ? "bottom-0" : "left-0",
        )}
        style={indicatorStyle}
        data-tabs-indicator
        aria-hidden="true"
      />
    </div>
  </TabsContext.Provider>

```

```
    );  
  },  
);
```

```
TabsRoot.displayName = "TabsRoot";
```

```
// 🌀 TABS LIST - Container for Triggers
```

```
export const TabsList = React.forwardRef<HTMLDivElement, TabsListProps>(  
  ({ children, className, showAddButton = false, onAddTab, ...props }, ref) => {  
    const { orientation, activationMode, value, onValueChange, triggerRefs } =  
      useTabsContext();
```

```
    const triggerRefsArray = useMemo(  
      () =>  
        Array.from(triggerRefs.entries()).map(([value, ref]) => ({  
          value,  
          ref,  
        })),  
      [triggerRefs],  
    );
```

```
    const handleKeyDown = useCallback(  
      (event: React.KeyboardEvent) => {  
        const triggers = triggerRefsArray.filter(  
          ({ ref }) => ref && !ref.disabled,  
        );  
        if (triggers.length === 0) return;
```

```
        const currentIndex = triggers.findIndex(({ value: v }) => v === value);  
        let nextIndex = currentIndex;
```

```
        const isHorizontal = orientation === "horizontal";
```

```
        switch (event.key) {  
          case "ArrowRight":  
            if (isHorizontal) {  
              event.preventDefault();  
              nextIndex = (currentIndex + 1) % triggers.length;  
            }  
            break;  
          case "ArrowLeft":  
            if (isHorizontal) {  
              event.preventDefault();  
              nextIndex =  
                (currentIndex - 1 + triggers.length) % triggers.length;  
            }  
            break;  
          case "ArrowDown":  
            if (!isHorizontal) {  
              event.preventDefault();
```

```

        nextIndex = (currentIndex + 1) % triggers.length;
    }
    break;
case "ArrowUp":
    if (!isHorizontal) {
        event.preventDefault();
        nextIndex =
            (currentIndex - 1 + triggers.length) % triggers.length;
    }
    break;
case "Home":
    event.preventDefault();
    nextIndex = 0;
    break;
case "End":
    event.preventDefault();
    nextIndex = triggers.length - 1;
    break;
default:
    return;
}

const nextTrigger = triggers[nextIndex];
if (nextTrigger && nextTrigger.ref) {
    nextTrigger.ref.focus();
    if (activationMode === "automatic") {
        onChange(nextTrigger.value);
    }
}
},
[orientation, activationMode, value, onChange, triggerRefsArray],
);

return (
    <div
        ref={ref}
        role="tablist"
        aria-orientation={orientation}
        className={cn(
            "inline-flex items-center gap-1",
            "bg-muted p-1 rounded-surface",
            orientation === "horizontal" ? "flex-row" : "flex-col",
            className,
        )}
        data-tabs-list
        onKeyDown={handleKeyDown}
        {...props}
    >
        {children}
        {showAddButton && (

```

```

    <button
      type="button"
      onClick={onAddTab}
      className={cn(
        "flex items-center justify-center",
        "p-1.5 rounded-[calc(var(--radius)-2px)]",
        "text-text-muted hover:text-text-main",
        "hover:bg-accent/50",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
        orientation === "horizontal" ? "ml-auto" : "mt-auto",
      )}
      aria-label="Add new tab"
    >
      <Plus className="w-4 h-4" aria-hidden="true" />
    </button>
  )}
</div>
);
},
);

```

```
TabsList.displayName = "TabsList";
```

```

// ⚡ TABS TRIGGER - Individual Tab Button
export const TabsTrigger = React.forwardRef<
  HTMLButtonElement,
  TabsTriggerProps
>(
  (
    {
      value,
      children,
      disabled = false,
      isCloseable = false,
      onClose,
      className,
      ...props
    },
    ref,
  ) => {
    const {
      value: currentValue,
      onValueChange,
      orientation,
      activationMode,
      registerTrigger,
      unregisterTrigger,
    } = props;
    return (
      <button
        type="button"
        className={cn(
          "flex items-center justify-center",
          "p-1.5 rounded-[calc(var(--radius)-2px)]",
          "text-text-muted hover:text-text-main",
          "hover:bg-accent/50",
          "transition-all duration-150 ease-out",
          "active:scale-95",
          "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
          orientation === "horizontal" ? "ml-auto" : "mt-auto",
        )}
        aria-label="Add new tab"
      >
        <Plus className="w-4 h-4" aria-hidden="true" />
      </button>
    );
  }
);

```

```

    activeTriggerRef,
    setActiveTriggerRef,
  } = useTabsContext();

const isActive = currentValue === value;
const triggerRef = useRef<HTMLButtonElement>(null);

const composedRef = useCallback(
  (node: HTMLButtonElement | null) => {
    triggerRef.current = node;
    registerTrigger(value, node);
    if (isActive) {
      setActiveTriggerRef(node);
      activeTriggerRef.current = node;
    }
    if (ref) {
      if (typeof ref === "function") {
        ref(node);
      } else {
        ref.current = node;
      }
    }
  },
  [value, registerTrigger, isActive, setActiveTriggerRef, ref],
);

useEffect(() => {
  return () => unregisterTrigger(value);
}, [value, unregisterTrigger]);

useEffect(() => {
  if (isActive && triggerRef.current) {
    setActiveTriggerRef(triggerRef.current);
    activeTriggerRef.current = triggerRef.current;
  }
}, [isActive, setActiveTriggerRef]);

const handleClick = useCallback(() => {
  if (!disabled) {
    onValueChange(value);
  }
}, [disabled, onValueChange, value]);

const handleFocus = useCallback(() => {
  if (activationMode === "automatic" && !disabled) {
    onValueChange(value);
  }
}, [activationMode, disabled, onValueChange, value]);

const handleClose = useCallback(

```



```

(e: React.MouseEvent) => {
  e.stopPropagation();
  onClose?.(value);
},
[onClose, value],
);

return (
  <button
    ref={composedRef}
    role="tab"
    aria-selected={isActive}
    aria-controls={`tabs-content-${value}`}
    id={`tabs-trigger-${value}`}
    tabIndex={isActive ? 0 : -1}
    disabled={disabled}
    type="button"
    className={cn(
      "relative z-10 flex items-center justify-center gap-1.5",
      "font-medium text-sm leading-none",
      "rounded-[calc(var(--radius)-2px)]",
      "transition-all duration-150 ease-out",
      "active:scale-95",
      "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring",
      "focus-visible:ring-offset-2 focus-visible:ring-offset-background",
      "disabled:pointer-events-none disabled:opacity-50",
      "disabled:cursor-not-allowed",
      "whitespace-nowrap",
      "orientation === \"horizontal\"
        ? \"px-4 py-2\"
        : \"px-3 py-2.5 w-full text-left\",
      isActive
        ? \"text-brand-primary font-semibold\"
        : \"text-text-main hover:bg-accent/50 hover:text-text-main\",
      isCloseable && \"pr-6\",
      className,
    )}
    onClick={handleClick}
    onFocus={handleFocus}
    {...props}
  >
    {children}
    {isCloseable && (
      <button
        type="button"
        onClick={handleClose}
        className={cn(
          "absolute right-2 top-1/2 -translate-y-1/2",
          "flex items-center justify-center",
          "p-0.5 rounded-[calc(var(--radius)-2px)]",

```

```

        "text-text-muted hover:text-text-main",
        "hover:bg-accent/50",
        "transition-all duration-150 ease-out",
        "active:scale-95",
        "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
        "disabled:pointer-events-none disabled:opacity-50",
    ))
    aria-label={`Close tab ${value}`}
    tabIndex={-1}
  >
    <X className="w-3.5 h-3.5" aria-hidden="true" />
  </button>
))
</button>
);
},
);

```

```

TabsTrigger.displayName = "TabsTrigger";

```

```

// 🌀 TABS CONTENT - Tab Panel

```

```

export const TabsContent = React.forwardRef<HTMLDivElement, TabsContentProps>(
  ({ value, children, forceMount = false, className, ...props }, ref) => {
    const { value: currentValue } = useTabsContext();

    const isActive = currentValue === value;

    if (!forceMount && !isActive) {
      return null;
    }

    return (
      <div
        ref={ref}
        role="tabpanel"
        id={`tabs-content-${value}`}
        aria-labelledby={`tabs-trigger-${value}`}
        hidden={!isActive}
        tabIndex={0}
        className={cn(
          "mt-4 focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2",
          "transition-all duration-200 ease-out",
          "opacity-0 data-[state=active]:opacity-100",
          "translate-y-1 data-[state=active]:translate-y-0",
          className,
        )}
        data-state={isActive ? "active" : "inactive"}
        data-tabs-content

```

```
        {...props}
      >
        {children}
      </div>
    );
  },
);
```

```
TabsContent.displayName = "TabsContent";
```

```
// 🌀 COMPOUND COMPONENT EXPORTS
```

```
export const Tabs = Object.assign(TabsRoot, {
  List: TabsList,
  Trigger: TabsTrigger,
  Content: TabsContent,
  Root: TabsRoot,
});
```

```
export type TabsOrientation = "horizontal" | "vertical";
export type TabsActivationMode = "automatic" | "manual";
```